CISC 322
Assignment 2

Concrete Architecture Analysis of Gemini CLI

March 13, 2026

The Unemployed
Gabrielle Garey (22dny@queensu.ca)
Calvin Wong (23dt5@queensu.ca)
Kesiya Jacob (22ccf5@queensu.ca)
Sofiia Bushareb (22wx56@queensu.ca)
Ryan Walsh (22bfr3@queensu.ca)
Grisha Tyukin (22ld34@queensu.ca)

Table of Contents

Table of Contents 2
1. Abstract 3
2. Introduction and Overview 3
3. Derivation Process 3
4. High-Level Architecture 5
- **4.1 Conceptual Architecture** 5
- **4.2 Concrete Architecture** 6
- **4.3 External Interfaces** 7
- **4.4 Reflexion Analysis** 8

5. Second-Level Subsystem Architecture 9
- **5.1 Conceptual Architecture** 9
- **5.2 Concrete Architecture** 10
- **5.3 Reflexion Analysis** 11

6. Use Cases 12
- **6.1 Use Case 1: Tool Request** 12
- **6.2 Use Case 2: Direct Response** 13

7. Limitations & Lessons Learned 14
8. Conclusion 16
9. Data Dictionary 16
10. Naming Conventions 16
11. AI Collaboration Report 17
12. References 18

1. Abstract

This report examines the concrete architecture of Gemini CLI, Google's open-source terminal-based AI assistant, building on the previously identified conceptual architecture. Through our analysis of the source code using the Understand tool, we identified subsystem dependencies and constructed a representation of the system's concrete architecture. The concrete architecture revealed additional dependencies not seen in the conceptual model, such as interactions between the Input Handler and Packages/CLI, the Tool Execution Framework and Core, and Task Delegation with both Core and Tools. We also identified the internal structure of the Tool Execution Framework, detailing components such as the Tool Scheduler, Execution Engine, and Result Handler. Reflexion analysis highlighted differences between the conceptual and concrete architecture, illustrating how recent features like agent skills affect the system. External interfaces, including Gemini CLI, MCP servers, and A2A servers, were also analyzed to understand how the system communicates with local and remote tools.

2. Introduction and Overview

Gemini CLI is Google's open-source AI assistant that connects directly to your local environment and can perform actions such as executing shell commands, searching the web, and reading or modifying files while maintaining awareness of the project's context (Gemini CLI Documentation, 2026). For this report, we were given the responsibility of using the Understand software development tool to analyze the concrete architecture of Gemini CLI.

To study the system, we examined the Gemini CLI codebase with Understand to identify subsystem dependencies. From this analysis, we constructed both a high-level architecture for Gemini CLI and a second-level architecture that highlights the internal structure of the Tool Execution Framework.

This report compares the conceptual architecture developed in the previous assignment with the concrete architecture derived from the source code. Reflexion analyses are used to determine how closely the implementation matches the conceptual design. In particular, we identify discrepancies where dependencies appear in the code but were not predicted in the conceptual model, and we discuss possible reasons for these discrepancies.

Our derivation process and how we arrived at these theories are discussed in the first section of our report. The report continues with an overview of the system's high-level architecture, followed by a more detailed analysis of the core subsystem to reveal the second-level architecture. Using these results, we examine two use cases, Tool Request and Direct Response, and update the sequence diagrams from the conceptual report to reflect the interactions and function calls observed in the actual implementation. The report concludes with a discussion of lessons learned and supporting materials, including the data dictionary, naming conventions, and AI collaboration report.

3. Derivation Process

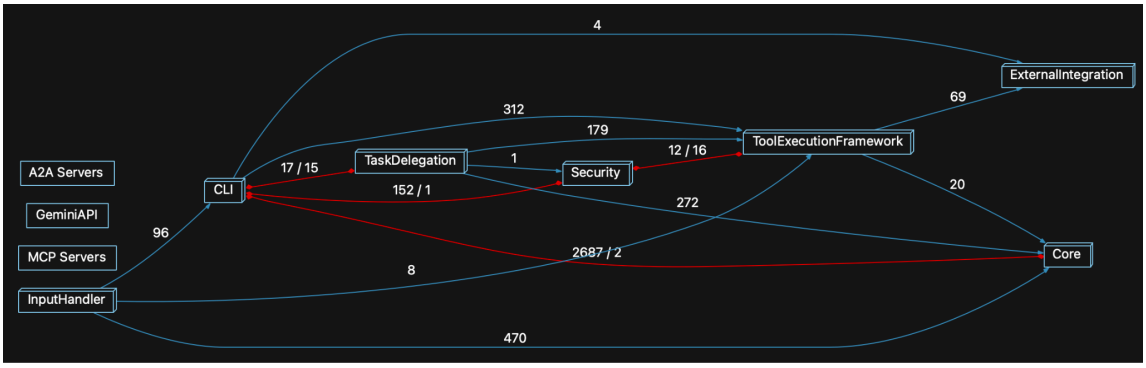

Figure 1: Concrete Architecture Dependency Diagram

To construct the concrete architecture of Gemini CLI, the derivation process involved examining the source code and dependency graph in Understand. We began by exploring the overall structure of the code and identifying how files and components were organized within the system. These elements were then compared with the conceptual architecture developed in Assignment 1 to determine how well the original design matched the actual implementation.

During this analysis, we reviewed the dependency graph generated in Understand and inspected relationships between files, components, and subsystems. We concluded that in our original conceptual architecture, certain dependencies and structural details were not represented clearly. In particular, some components interacted across subsystem boundaries or contained responsibilities that were not initially predicted.

To build the concrete architecture, we focused on the main packages identified in the source code, including the core package. By analyzing the code, function calls, and component dependencies within these packages, we identified key subsystems and grouped related components together. This allowed us to construct a concrete architectural view that accurately represents how Gemini CLI is implemented and how its components interact in practice.

4. High-Level Architecture

4.1 Conceptual Architecture

The box and line diagram is based on the conceptual architecture we developed in our Assignment 1. In that assignment, we identified the main components involved in processing user requests and interacting with external services. Building on that analysis, the diagram refines the design by organizing the system into key subsystems.

The design separates the system into client-side components and server-side components. User interaction begins on the client side through the CLI, while the main processing and coordination logic occurs on the server side. The architecture is organized around a central core component that coordinates external services and internal subsystems (Garey et al., 2026).

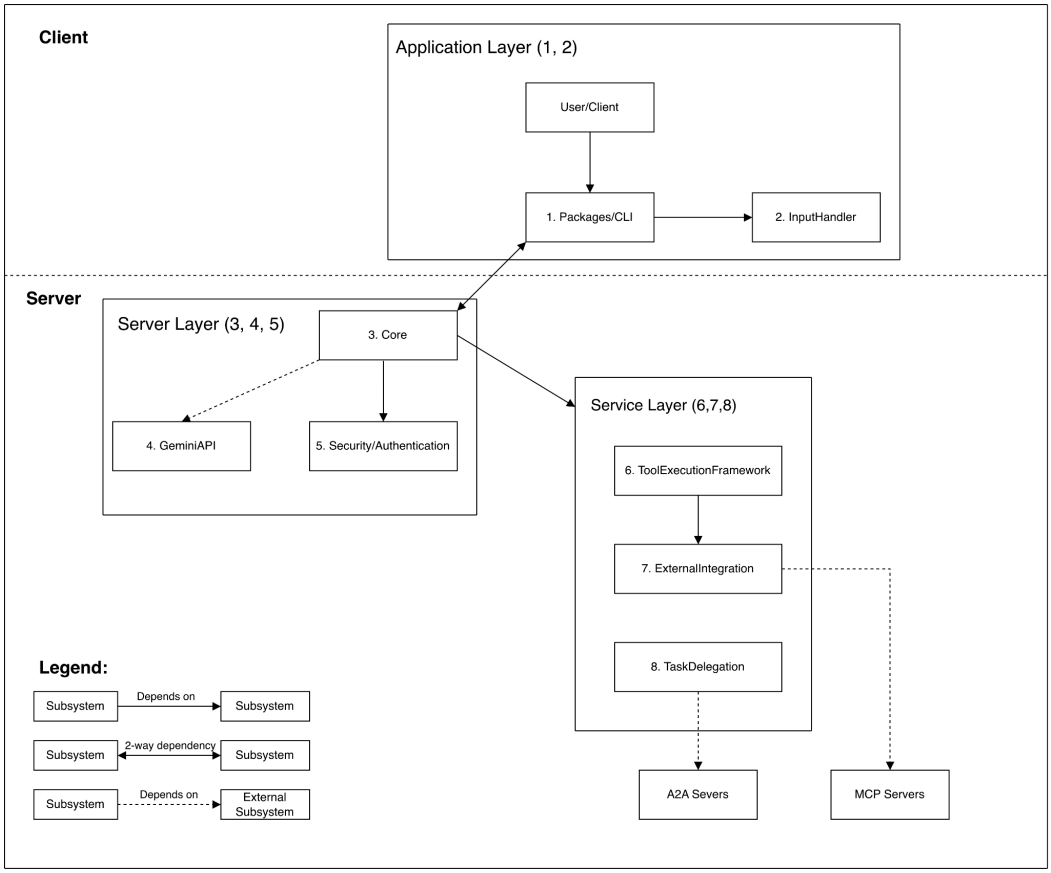

Figure 2: Conceptual Architecture of Gemini CLI

Compared to the conceptual architecture presented in the previous report, this diagram in Figure 2 provides a more concrete representation of the system's structure and dependencies. The earlier design described the system using a client–server architecture that focused on the high-level separation between the user interface and the backend processing components. In this report, the architecture is refined to show specific subsystems and their interactions within the server side of the system. It also explicitly illustrates integrations with external services such as the Gemini API, A2A servers, and MCP servers, which were not fully detailed in the conceptual design. These additions clarify how the CLI coordinates AI model requests, tool execution, and communication with external systems. In developing more detailed architectural representation, we adopted a layered architectural style inspired by Group 8's approach.

4.2 Concrete Architecture

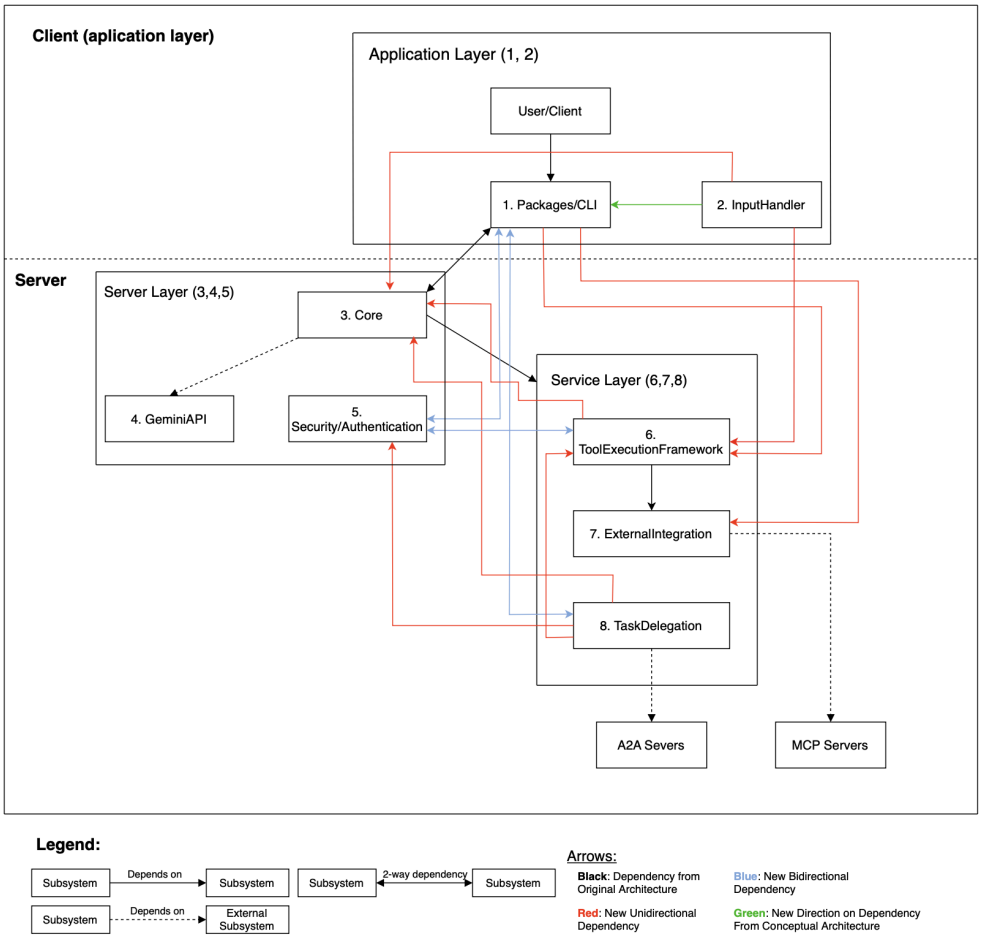

Figure 3: High-Level Concrete Architecture of Gemini CLI

With the help of the dependency graph created using Understand, we were able to produce the concrete architecture shown in Figure 3. Compared with our conceptual architecture (Figure 2), the concrete architecture shows many more dependencies between components. We will explain these new dependencies in more detail later.

- ***The User/Client*** represents the person interacting with the system through the command-line interface. The user submits commands or requests that are processed by the Packages/CLI component. This part of the architecture exists to provide the entry point for interaction with the system.
- ***The Packages/CLI subsystem*** is responsible for handling command-line execution. It receives user commands and prepares them for processing by the system. This component also communicates with the Input Handler to interpret the structure of user commands. The CLI exists to provide a structured interface between the user and the internal components of the system.

- ***The InputHandler subsystem*** processes and interprets user input from the CLI. It parses commands, validates parameters, and prepares the request before passing it to the core system logic. This subsystem exists to ensure that user commands are correctly formatted and understandable by the rest of the system.
- ***The Core subsystem*** acts as the central controller of the architecture. It coordinates the overall workflow of the system and determines how requests should be processed. The Core communicates with multiple components including the Gemini API, the Security/Authentication subsystem, and the Service Layer. This subsystem exists to manage the interaction between different parts of the system and ensure that requests are processed correctly.
- ***The GeminiAPI subsystem*** represents the external AI model used by the system. The Core subsystem sends requests to this external service when AI-generated responses or model-based processing is required. This component exists to provide advanced language-model capabilities that are not implemented locally in the system.
- ***The Security/Authentication subsystem*** is responsible for verifying permissions and ensuring secure communication within the system. It handles authentication tokens and policy enforcement when the Core subsystem interacts with other services. This subsystem exists to protect system resources and ensure that only authorized operations are performed.
- ***The ToolExecutionFramework subsystem*** manages the execution of local tools such as file operations or shell commands. When the Core determines that a task requires a local tool, the request is sent to this subsystem. This subsystem exists to isolate tool execution and ensure that tools are executed in a controlled environment.
- ***The ExternalIntegration subsystem*** allows the system to communicate with external MCP servers. These servers provide additional tools and services that are not directly part of the local system. This subsystem exists to extend the system's functionality by enabling integration with external tool providers.
- ***The TaskDelegation subsystem*** is responsible for sending complex tasks to external A2A servers. These servers can perform distributed processing or specialized operations. This subsystem exists to support scalability by allowing certain tasks to be handled outside the core system.
- ***The A2A Servers and MCP Servers*** represent external services used by the system. A2A servers are used for delegated task execution, while MCP servers provide external tools and integrations. These external systems extend the capabilities of the architecture beyond the local environment.

4.3 External Interfaces

There are three external interfaces in the concrete architecture, each serving a distinct purpose. GeminiAPI connects the CLI to Google's Gemini AI model, allowing it to send prompts and receive AI-generated responses. MCP Servers act as a bridge between Gemini CLI and external tools and services. The ToolExecutionFramework has built-in MCP integration, where *mcp-client.ts* manages the connection to MCP servers directly. A2A Servers allow Gemini CLI to communicate with other AI agents, enabling it to delegate tasks to specialized sub-agents.

4.4 Reflexion Analysis

The concrete architecture of Gemini CLI shows multiple unexpected dependencies that were not present in the conceptual design. Many of these stem from the recent addition of agent and skills features. We analyzed the divergences by looking at the source code using Understand and git.

InputHandler → Packages/CLI

A divergence from the conceptual architecture is InputHandler's dependency on Packages/CLI. Specifically, *skillUtils.ts* in InputHandler imports *SettingScope* from *settings.js* in Packages/CLI. This was added with the commit message "[Skills] Foundation: Centralize management logic and feedback rendering." by NTaylorMullen two months ago (*google-gemini/gemini-cli*, n.d.). *SettingScope* is an enum defined in CLI's settings module that represents the levels at which a configuration setting can be applied, such as user-wide or workspace-specific. Within *skillUtils.ts*, *SettingScope* is used inside *renderSkillActionFeedback* to format scope labels when generating user-facing feedback messages about skill enable and disable actions. For example, distinguishing whether a skill was modified at the workspace or user level.

ToolExecutionFramework → Core

Another divergence from the conceptual architecture is ToolExecutionFramework's dependency on Core. In particular, *skillLoader.ts* in ToolExecutionFramework imports *debugLogger* from Core, added with the commit message "Agent Skills: Unify Representation & Centralize Loading." by NTaylorMullen two months ago (*google-gemini/gemini-cli*, n.d.). *debugLogger* is a shared logging utility that records error messages for developers when something goes wrong at runtime. Within *skillLoader.ts*, it is used to report problems that arise while finding and loading skill files.

TaskDelegation → ToolExecutionFramework

An additional divergence from the conceptual architecture is TaskDelegation's dependency on ToolExecutionFramework. Notably, *agentLoader.ts* in TaskDelegation imports *isValidToolName* from *tool-names.ts* in ToolExecutionFramework. This was added with the commit message "Refactor subagent delegation to be one tool per agent" two months ago by gundermanc (*google-gemini/gemini-cli*, n.d.). *isValidToolName* is a function that checks whether a given tool name is valid. Within *agentLoader.ts* it is used to validate the tool names listed in an agent's definition file when the agent is being loaded. This ensures that any tools an agent is configured to use actually exists in the system.

TaskDelegation → Core

Another divergence from the conceptual architecture is TaskDelegation's dependency on Core. Specifically, *agentLoader.ts* in TaskDelegation imports getErrorMessage from error.ts in Core. This was added with the commit message "feat(agents): implement first-run experience for project-level sub-agents." two months ago by gundermanc (*google-gemini/gemini-cli*, n.d.). *getErrorMessage* is a utility function in Core that converts raw error objects into readable error messages. Within *agentLoader.ts*, it is used to generate human-readable descriptions of errors that occur when agent definition files cannot be read or parsed.

5. Second-Level Subsystem Architecture

The second-level subsystem we are analyzing is the tool execution framework in Gemini CLI. This component's job is to provide an extensible tool execution system for model capabilities. Our conceptual architecture divides the tool execution framework into the subsystems: tool dispatch, tool registry, permission controller, execution engine, and result handler. We aimed to model how these components would interact with each other and the other major components of Gemini CLI to ensure safe and precise tool use. The conceptual architecture for the subsystem also reinforced our belief that Gemini CLI uses a pipe and filter architecture, when executing tools in the Packages/CLI.

5.1 Conceptual Architecture

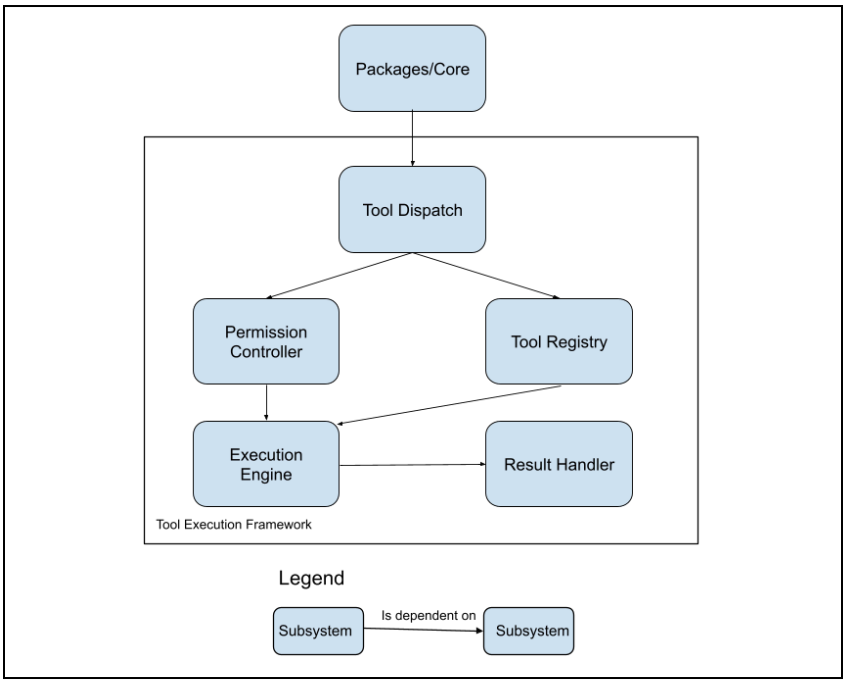

Figure 4: Low-Level Conceptual Architecture of the Tool Execution Framework in Gemini CLI

5.2 Concrete Architecture

We created the concrete architecture by building an architectural model in Understand, then assigning the files we identified as part of the tool execution framework to a component from our conceptual architecture. We then generated a dependency graph to verify our original analysis, and that, along with an examination of the code, allowed us to identify 2 additional components needed to complete the architecture, as well as dependencies that had been overlooked.

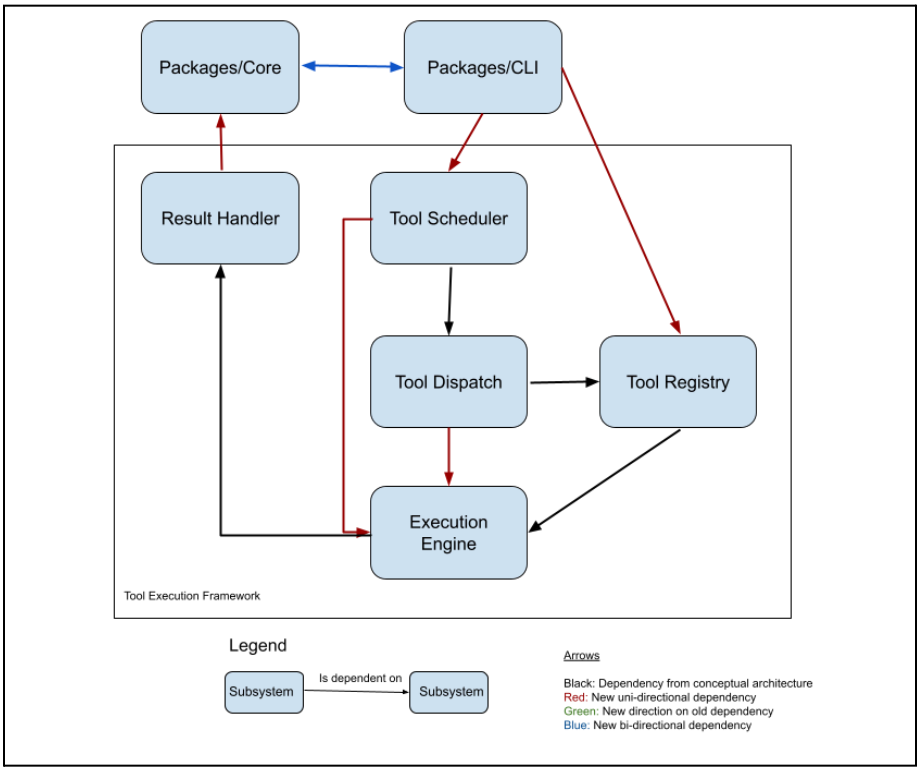

Figure 5: Low-Level Concrete Architecture of the Tool Execution Framework in Gemini CLI

Based on the conceptual and concrete architectures, the components are defined as follows:

- ***Tool Scheduler*** is a component that manages a queue of requests from the orchestration engine. It is responsible for deciding what tool will be executed next, and deals with the coordination between the tool execution framework and tool use requests from Gemini.
- ***Tool Dispatch*** is responsible for routing tool call requests to the correct tool implementation. It consults the tool registry to look up the requested tool by unique identifier, invokes it, and forwards the result to the tool response handler to be returned to the orchestration engine or the CLI
- ***Permission Controller*** Upon inspection of the code, we determined that permission controller is not a concrete component. Concretely, it's a method (`shouldConfirmExecute()`) that is distributed across each tool's BaseTool implementation. That is to say, it is coded directly into the tool registry. We decided to keep it as a visual component to show that a check is made for permission.
- ***Tool Registry*** is what manages the available tools of Gemini CLI. It includes data about each tool, such as name, description, input schema, required permissions, etc., and supports both static tools like command line tools and registered tools from external servers like the MCP servers. It is consulted by the toolDispatch module when routing a tool call request and by the permissions controller that verifies that the request has the authorization to use the tool.
- ***Execution Engine*** is what actually manages setting up the environment and executing the tool.

- ***Result Handler*** is tasked with processing and formatting the output of an executed tool, received from the Tool Dispatcher, before returning it to the orchestration engine. It ensures the result is structured correctly for the Gemini API, and handles error cases where a tool execution fails or returns an unexpected output. It is also responsible for returning the result to the appropriate component in the high-level architecture.

5.3 Reflexion Analysis

The following are divergences from the conceptual architecture found in the concrete architecture.

Tool Dispatch → Execution Engine
A divergence from the conceptual architecture is Tool Dispatch's dependency on Execution Engine. Specifically, *scheduler.test.ts* in Tool Dispatch imports *ToolConfirmationOutcome* and *AnyDeclarativeTool* from *tools.ts* in Execution Engine. This was added with the commit message "refactor(core): decouple scheduler into orchestration, policy, and confirmation" by abhipatel12 two months ago (*google-gemini/gemini-cli*, n.d.). *ToolConfirmationOutcome* is an enum defined in tools.ts that represents the possible outcomes of a user confirmation prompt, such as *ProceedOnce*, *ProceedAlways*, or *Cancel*. Within *scheduler.test.ts*, it is used in the Phase 3 confirmation tests to assert on user decisions such as *Cancel* and *ProceedAlways*. *AnyDeclarativeTool* is used to type mock tool objects in fabricated tool call states like *ValidatingToolCall* and *ExecutingToolCall*.

Tool Scheduler → Execution Engine
Another divergence from the conceptual architecture is Tool Scheduler's dependency on Execution Engine. Specifically, *coreToolScheduler.ts* in Tool Scheduler imports *ToolConfirmationOutcome*, *AnyDeclarativeTool*, *AnyToolInvocation*, *ToolCallConfirmationDetails*, and *ToolConfirmationPayload* from *tools.ts* in Execution Engine. This was added with the commit message "refactor(core): extract static concerns from CoreToolScheduler" by abhipatel12 three months ago (*google-gemini/gemini-cli*, n.d.). *ToolConfirmationOutcome* is an enum representing the possible outcomes of a user confirmation prompt, such as *ProceedOnce*, *ProceedAlways*, or *Cancel,* and is used to set tool call outcomes, handle confirmation responses, and mark cancelled calls. *AnyDeclarativeTool* and *AnyToolInvocation* are type aliases that represent a tool and its executable invocation without committing to specific parameter or result types, and are used to type tool objects held in active tool call state. *ToolCallConfirmationDetails* and *ToolConfirmationPayload* are used to type the confirmation UI details and any user-submitted data passed through the confirmation flow in *handleConfirmationResponse*.

Packages/CLI → Core
An additional divergence from the conceptual architecture is the dependency of Packages/CLI on Core. Specifically, *config.integration.ts* in Packages/CLI imports *Config* and *DEFAULT_FILE_FILTERING_OPTIONS* from *config.ts* in Core. This was added with the commit message "fix(cli) : fixed bug #8310 where /memory refresh will create discrepancies with initial memory load ignoring settings/config for trusted folder and file filters (#10611)" by sgnagnarella and bdmorgan five months ago (*google-gemini/gemini-cli*, n.d.). *Config* is the

central configuration class in Core that holds all runtime settings for the CLI. *DEFAULT_FILE_FILTERING_OPTIONS* is an object defining default values for file filtering behaviour, such as whether to respect *.gitignore* files. In *config.integration.test.ts*, *Config* is instantiated with various *ConfigParameters* to verify settings are correctly applied, and *DEFAULT_FILE_FILTERING_OPTIONS* is used as the expected baseline in tests that check behaviour when no custom file filtering is provided.

Packages/CLI → Tool Registry
Another divergence from the conceptual architecture is the dependency of Packages/CLI on Tool Registry. Specifically, *atCommandProcessor.test.ts* in Packages/CI imports *ToolRegistry* from *tool-registry.ts* in Tool Registry. This was added with the commit message "Fix windows bugs in atCommandProcessor.ts" by scidomino eight months ago (*google-gemini/gemini-cli*, n.d.). *ToolRegistry* is the class responsible for registering, managing, and retrieving tools available to the CLI. In *atCommandProcessor.test.ts*, a real *ToolRegistry* instance is constructed in the test setup and populated with *GlobTool* and *ReadManyFilesTool* instances, so that *handleAtCommand* can resolve *@file* references using real tool execution rather than mocks.

6. Use Cases

6.1. Use Case 1: Tool Request

The user types a prompt in the terminal requesting file creation (e.g., gemini "fix the bug in parser.py"). This prompt is received by *submitQuery()* in *useGeminiStream.ts.* In the Packages/CLI component. Before forwarding the input, *submitQuery()* calls *prepareQueryForGemini()* to preprocess the query. Once a valid query is ready, *submitQuery()* calls *geminiClient.sendMessageStream()*, handing off to the Core package and beginning the API request cycle. The Core sends the request to GeminiAPI via *generateContentStream()*, which is defined in the *ContentGenerator* interface in *contentGenerator.ts*. GeminiAPI processes the prompt and determines that the write_file tool is needed to fulfill the request. The API returns a tool use request to the Core component. The Core subsystem then forwards this request to ToolExecutionFramework, specifically to *write-file.ts*. Within this file, the *WriteFileTool* implementation is selected. It creates a *WriteFileToolInvocation* object that represents a specific execution of the tool with the parameters provided by the model. Before the tool can execute, the request is evaluated by the Security/Authentication component. This is done in the *check()* method in *policy-engine.ts*, which evaluates the tool invocation against the configured security policies. During this evaluation, *ruleMatches()* is used to determine whether any policy rule applies to the tool call. The *check()* method returns ASK_USER, indicating user approval is required. The ToolExecutionFramework receives this decision and calls *getConfirmationDetails()* on the *WriteFileToolInvocation* object, which produces the file path and a diff of the proposed changes. This information is passed back to Packages/CLI, where it is displayed to the user. If the user approves, the ToolExecutionFramework calls *execute()* on the *WriteFileToolInvocation*, which writes the file to the file system.

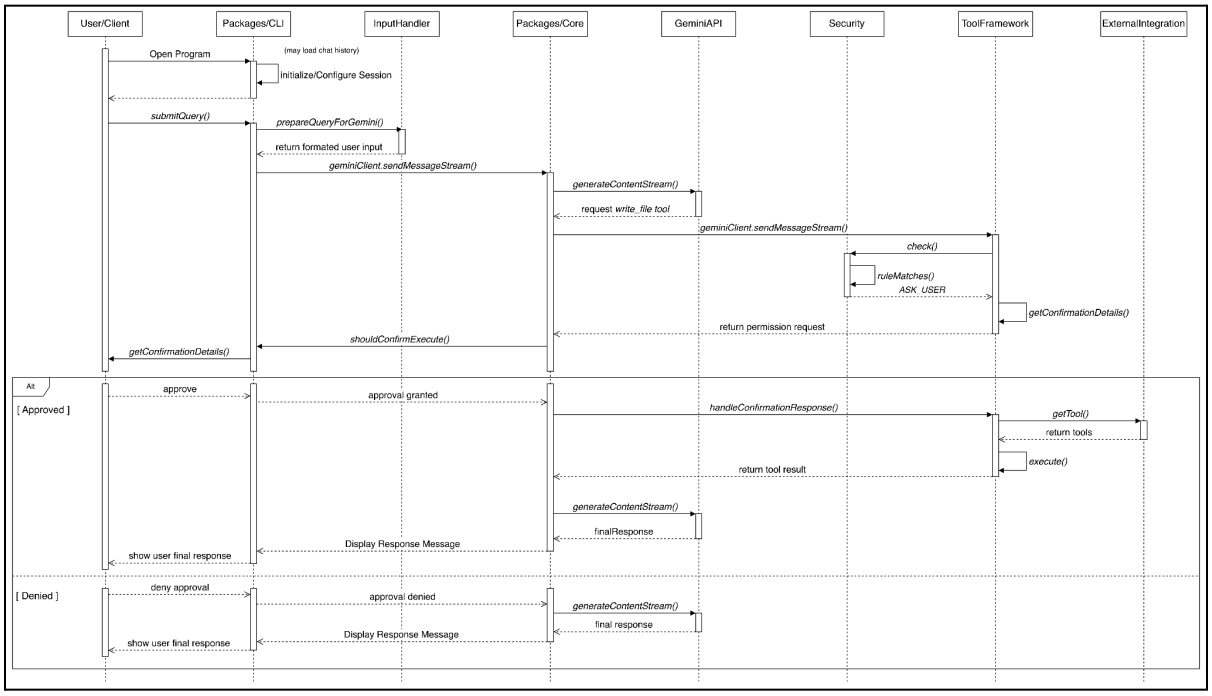

Figure 6: Tool Request Sequence Diagram

6.2. Use Case 2: Direct Response

The user types a prompt in the terminal requesting an explanation (e.g., gemini "explain how list comprehensions work in Python"). This prompt is received by *submitQuery()* in *useGeminiStream.ts* in the Packages/CLI component. Before forwarding the input, *submitQuery()* calls *prepareQueryForGemini()* to preprocess the query. Once a valid query is ready, *submitQuery()* calls *geminiClient.sendMessageStream()*, handing off to the Core package and beginning the API request cycle. GeminiAPI processes the prompt and generates an explanation as a direct text response, with no tool use required. The API returns the text response to the Core, which forwards it back to Packages/ CLI. Packages/CLI then formats and displays the explanation in the terminal for the user to read.

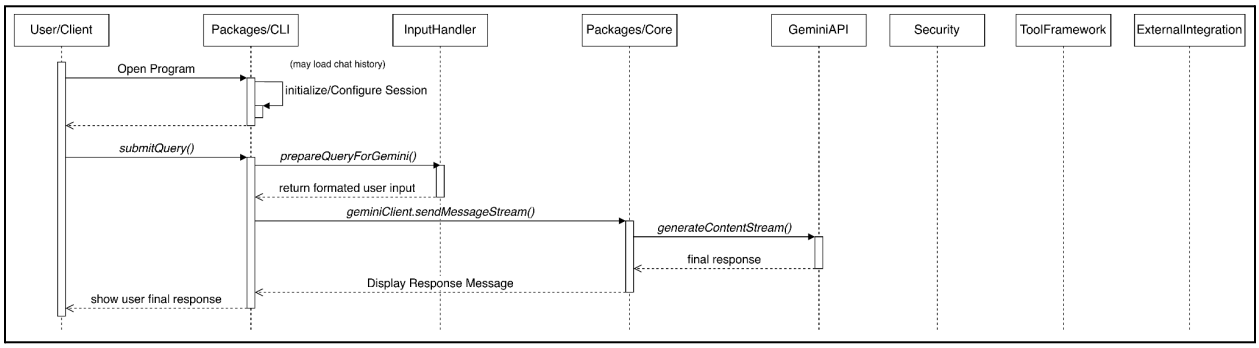

Figure 7: Direct Response Sequence Diagram

7. Limitations & Lessons Learned

After exploring both the conceptual and concrete architectures for Gemini CLI, we have gained an appreciation for the difficulty in bridging the gap. Initially, we tried to keep the structure of our concrete architecture as close as possible to our conceptual design from assignment 1. However, after our analysis with the help of the Understand Tool, we realized our initial conceptual architecture was far from adequate. We decided to overhaul a significant portion of our conceptual architectural design, which helped us to create a far more accurate concrete representation of the system. We learned that being willing to restructure/change when needed, even if it takes more time, is important if we want to be accurate and correct.

Additionally, we learned how the quality and clarity of reports and diagrams impact our understanding of the system. We used parts of group 8's (Kollipara et al., 2026). conceptual architecture report to modify our own. This group had great diagram descriptions and clarity, which made understanding their conceptual architecture far easier, especially compared to other resources we consulted with worse diagram clarity. Having been able to consider this group's architecture proved significant in refining our architectural designs.

We found the most significant limitation we faced was deciphering the scope of components. There were many test files for components with different dependencies from their corresponding files. Since these test files would often reach into different dependencies throughout the system, we often found it confusing and difficult to decipher the actual scope of components and where their dependencies lay. Another limitation of our analysis was our limited knowledge of TypeScript, which made it more difficult to identify and interpret dependency relationships and to trace how specific use cases were implemented across the codebase.

8. Conclusion

In conclusion, our report provided an in-depth analysis of Gemini CLI's concrete architecture from a high-level system view all the way down to inner subsystems and components. Using the Understand Tool, we were able to analyze the system's dependencies, relationships, and execution details, which helped us build upon our conceptual architecture from assignment 1. This led us to determine that Gemini CLI operates on a primarily client-server and layered architecture style.

We explored the high-level Tool Execution Framework subsystem in greater detail by examining its internal components and dependencies. We performed a reflexion analysis to investigate discrepancies between our conceptual and concrete architecture designs, analyzing differences between the two box-and-arrow diagrams, which helped broaden our understanding of the system's actual architecture.

Our experience examining source code, interpreting dependencies, and determining the distribution of responsibilities across the Gemini CLI system gave us a deeper understanding of how the system operates. Overall, this assignment showed us that while the conceptual architecture is useful for a general understanding of a system's structure, concrete architecture is

vital for revealing the system interactions, coordination complexity, implementation details, and architectural trade-offs that exist in practice.

9. Data Dictionary

CLI Package (packages/cli): User-facing component that handles command-line interaction, processes user input, manages conversation history, and renders terminal output
Core Package (package/core): Backend component orchestrating communication with the Gemini API, managing prompt construction, session state, and tool execution
Tool Modules (packages/core/src/tools/): Extensions allowing Gemini CLI to interact with the local environment (file reading, shell commands, web content retrieval)
Tool Registration: Procedure for registering available tool modules in Core so they can be invoked by the Gemini model

10. Naming Conventions

CLI: Command Line Interface
Core: Backend service logic/business logic layer
NPM: Node Package Manager
MCP: Model Context Protocol (tool integration layer)
JSON: JavaScript Object Notation

11. AI Collaboration Report

For A2, our AI teammate was Perplexity (February 2026 version). We selected this tool because the assignment required concrete analysis supported by documentation, and Perplexity is particularly strong at retrieving source-linked technical information. Unlike a single-model assistant, Perplexity operates as an AI agent that can automatically select and route queries to different models such as GPT, Claude, or Gemini, depending on the task. This flexibility made it well-suited for our workflow, since different tasks such as retrieval, summarization, and critique benefited from different model strengths.

Tasks Assigned to the AI Teammate

Locate low-level component interactions in the GeminiCLI system.
Rationale: This task involved the review of hundreds of low-level system components to find interactions between them, which is incredibly time-consuming. Using our AI teammate to locate these interactions allowed us to narrow the scope significantly, leading to a more efficient workflow.

Summarize lengthy technical documentation into concise architecture-relevant takeaways.
Rationale: The sources were dense and technical, so using AI helped quickly produce structured summaries that we could then verify against the originals.

Suggest concrete architecture refinements by identifying missing implementation details in our analysis. Rationale: Since diagrams require precision, the agent helped detect where our explanation lacked implementation-level detail, such as removed dependencies or unclear component boundaries, allowing us to improve completeness efficiently.

Interaction Workflow

Our group used a collaborative workflow: each member drafted prompts for their section and shared both the prompts and the AI outputs with the group. Before sending prompts, we first discussed what we wanted to learn from the AI and gathered local context such as our current diagrams, assumptions, and assignment requirements. After receiving responses, we reviewed them together, decided what was useful, and edited or discarded anything unclear or unsupported.

One of our more complex prompts asked the AI to act like a software architecture TA and critique our proposed concrete architecture of Gemini CLI against an implementation-level breakdown. We provided context such as our subsystem decomposition, dependency findings from Understand, second-level Tool Execution Framework components, and the concrete use cases we were analyzing, including interactions among Packages/CLI, Core, Tool Execution Framework, Task Delegation, and external interfaces such as Gemini API, MCP servers, and A2A servers. We asked for a clear, structured response that identified missing dependencies, weak subsystem boundaries, and places where our explanation did not fully match the observed code relationships. As we looked more closely at the implementation, we found that broad prompts were not enough to support concrete architecture work. We then shifted to narrower, evidence-based prompts that focused on specific subsystems, dependency relationships, and code-level observations, which produced more useful feedback for revising our report.

Validation Procedures

All AI outputs were treated as drafts. We verified factual claims by checking primary sources, cross-referencing summaries with original documentation, and edited all text for clarity and accuracy. Any claim that could not be validated was removed or clearly labelled as an assumption.

Estimated Contribution

We estimate that the AI contributed approximately 12% of the final deliverable:

- 5% for locating low-level component interactions and narrowing the scope of implementation-level analysis.
- 4% for suggesting refinements to the concrete architecture by identifying missing dependencies, unclear subsystem boundaries, and other implementation-level gaps.
- 3% for summarizing technical documentation into concise, architecture-relevant takeaways.

All architectural decisions, interpretations, and diagrams remained human-generated.

Reflection on Team Dynamics

The AI improved efficiency in drafting, research, and brainstorming, but it also introduced additional verification work. It helped identify missing assumptions and improved clarity through rewriting support. However, some responses sounded confident even when partially inferred, so our group adopted a strict rule that factual claims required source validation. Overall, we learned that effective AI collaboration requires clear prompts, structured outputs, and consistent verification.

12. References

Garey, G., Wong, C., Jacob, K., Bushareb, S., Walsh, R., & Tyukin, G. (2026, February 13). *Conceptual Architecture Analysis of Gemini CLI*. https://gcgarey.github.io/CISC-322.github.io/static/Deliverable1.pdf

Gemini CLI documentation. (2026). Gemini CLI. https://geminicli.com/docs/

google-gemini. (2025). *GitHub - google-gemini/gemini-cli: An open-source AI agent that brings the power of Gemini directly into your terminal.* GitHub. https://github.com/google-gemini/gemini-cli

Kollipara, A., Smith, A., Fiorino, C., Amos, L., Turner, M., & Webster, V. (2026, February 13). *Gemini CLI Conceptual Architecture Analysis*. https://teamm8s.github.io/CISC322-Project-A1.pdf